

Git Branching Strategies, Pull Requests & Repository Workflows

A practical guide to how teams organize Git branches, review code, integrate changes, structure repositories, and verify commit identity.

1. Why Do We Need a Branching Strategy?

When one developer works alone, committing directly to `main` can be manageable.

```
A - B - C - D
          ↑
        main
```

In a team, many developers may be working at the same time:

- Developer A → Login
- Developer B → Payment
- Developer C → Search
- Developer D → Bug fix
- Developer E → Database migration

If everyone commits directly to `main`, the branch may contain a mixture of:

- incomplete features
- experimental code
- production fixes
- database changes
- broken functionality

The result can become:

```
main ≠ stable software
```

The Fundamental Problem

A development team has two competing needs:

```
Developers need freedom to change code
+
Production needs stability
```

Branches allow developers to work independently without immediately affecting the production-ready code.

```
main
|
├─ feature/login
├─ feature/payment
└─ bugfix/cart
```

But once branches exist, teams need rules for questions such as:

- Where should a branch be created from?
- Where should it be merged?
- When should it be merged?
- How long should it live?
- When should a release happen?
- How should production bugs be handled?

That set of rules is the **branching strategy**.

What Is a Branching Strategy?

A branching strategy is a set of rules a team follows for:

- creating branches

- using branches
- merging branches
- deleting branches
- preparing releases
- handling production fixes

Three important approaches are:

1. **Gitflow**
2. **GitHub Flow**
3. **Trunk-Based Development**

They solve similar problems, but with very different philosophies.

2. Gitflow

Gitflow is the most structured of the three approaches.

It fits software where releases are prepared as versions such as:

```
v1.0  
v1.1  
v1.2  
v2.0
```

The development lifecycle may look like:

```
Develop features  
↓  
Integrate them  
↓  
Test release  
↓  
Prepare version  
↓  
Deploy
```

Gitflow was designed around this kind of release process.

2.1 Permanent Branches in Gitflow

Gitflow mainly revolves around two long-lived branches:

```
main
develop
```

main

Represents production-ready or released code.

```
main
↑
Production-ready releases
```

Think of it as:

| What is currently considered production or release ready?

develop

Represents the latest integrated development for the next release.

```
develop
↑
Latest integrated development
```

Think of it as:

| What are we currently building for the next release?

2.2 Feature Branches

Suppose production currently contains version **1.0**.

New development for version **1.1** starts from **develop**.

Developers create separate feature branches:

```
develop
|
├─ feature/login
├─ feature/payment
└─ feature/search
```

Typical workflow:

```
git switch develop
git pull

git switch -c feature/payment
```

Developers work independently on the feature.

```
develop
A - B - C
      \
       D - E - F
           ↑
    feature/payment
```

Once the feature is complete, it is merged into `develop`.

```
feature/payment
  ↓
develop
```

Why Not Merge Directly Into `main` ?

Because:

```
Feature finished
```

does not necessarily mean:

Release ready

A release may require several features:

- Login
- Payment
- Search
- Notifications

These features can first be integrated into `develop`, while `main` remains stable.

2.3 Release Branches

Once the features required for a release are ready, Gitflow can create a release branch.

Example:

```
release/1.1
```

It is created from:

```
develop
```

Conceptually:

```
develop
|
└─ release/1.1
```

The release branch is mainly used for:

- testing
- release-specific bug fixes

- version number changes
- documentation
- release preparation

New features usually should not keep entering that release branch.

Meanwhile, development for future work can continue on `develop`.

Once the release is stable:

```
release/1.1
  ↓
main
```

The release changes are also normally synchronized back into `develop`.

2.4 Hotfix Branches

Suppose version `1.1` is already in production and a critical payment bug appears.

The team should not wait for unfinished work currently sitting in `develop`.

Gitflow uses:

```
hotfix/*
```

A hotfix normally starts from `main`.

```
main
|
└─ hotfix/payment-null-error
```

After fixing the bug, merge the hotfix into:

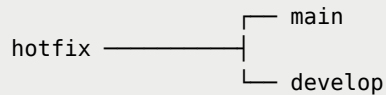
```
main
```

because production needs the fix immediately.

Also merge the fix into:

develop

Otherwise, the next release may accidentally reintroduce the same bug.



2.5 Gitflow Branch Summary

Branch	Purpose
main	Production/released code
develop	Integrated code for the next release
feature/*	Individual feature development
release/*	Preparing a release
hotfix/*	Emergency production fixes

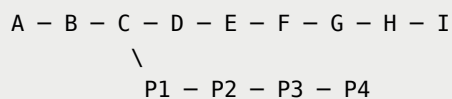
2.6 Main Problem With Gitflow

Gitflow can encourage **long-lived branches**.

Suppose:

feature/payment

exists for three weeks while `develop` keeps changing.



When the feature finally merges, the two histories may have diverged significantly.

That often creates:

- larger merge conflicts
- harder integrations
- stale feature branches
- delayed feedback

This leads to an important idea:

Maybe long-lived branches themselves are part of the problem.

3. GitHub Flow

GitHub Flow simplifies the workflow considerably.

It works well for applications that can be deployed frequently, such as:

- web applications
- SaaS products
- backend APIs
- startup products

Instead of preparing a release branch for every version, the workflow is closer to:

```
Feature ready
  ↓
Review
  ↓
Test
  ↓
Merge
  ↓
Deploy
```

3.1 Main Idea

`main` is the central branch.

Developers create short-lived branches from it.

```
main
|
├─ feature/login
├─ feature/payment
└─ fix/cart
```

Compared with Gitflow, the following usually disappear:

```
develop
release/*
```

That is the biggest simplification.

3.2 Typical GitHub Flow

Start from an updated `main`:

```
git switch main
git pull
```

Create a branch:

```
git switch -c feature/payment
```

Work and commit:

```
git add .
git commit -m "Add payment validation"
```

Push the branch:

```
git push -u origin feature/payment
```

Then open a Pull Request:

```
feature/payment
  ↓
main
```

Typical flow:

```
Branch
  ↓
Commits
  ↓
Push
  ↓
Pull Request
  ↓
Review
  ↓
CI checks
  ↓
Merge
  ↓
Delete branch
```

Important

GitHub Flow is not simply “using GitHub.”

It is a branching workflow built around:

- one healthy central branch
- short-lived branches
- Pull Requests
- reviews
- automated checks
- frequent merges

The core principle is:

Keep `main` healthy and use short-lived branches around it.

3.3 Limitation of GitHub Flow

GitHub Flow does not automatically prevent long-lived branches.

If:

```
feature/payment
```

stays open for two months, the same problem appears again:

```
main keeps moving
    ↓
feature branch stays separate
    ↓
divergence grows
    ↓
large integration later
```

Trunk-Based Development pushes the idea of frequent integration even further.

4. Trunk-Based Development

Trunk-Based Development is often misunderstood as:

| Everyone pushes random code directly to `main`.

That is not the core idea.

The actual philosophy is:

| Integrate everyone's changes into one shared trunk very frequently.

Usually:

```
trunk ≈ main
```

4.1 What Is Integration?

Suppose two developers branch from the same commit:

```
A - B - C
      ↑
    main
```

Developer A:

```
C - A1 - A2 - A3 - A4
```

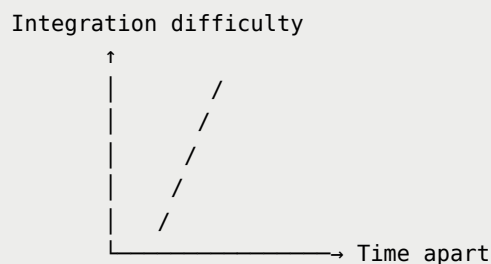
Developer B:

```
C - B1 - B2 - B3
```

If they work separately for a long time, their branches diverge.

Combining those independent changes again is **integration**.

The longer developers stay separated, the harder integration can become.



4.2 Trunk-Based Development's Answer

Instead of integrating once every few weeks:

```
Long development
↓
```

Huge integration

integrate small changes frequently:

```
Small change
↓
Integrate
↓
Small change
↓
Integrate
↓
Small change
↓
Integrate
```

The goal is to keep developers close to the shared trunk.

4.3 Does Trunk-Based Development Use Branches?

Yes.

Trunk-based does **not** necessarily mean “no branches.”

Approach 1: Direct commits to the trunk

Some teams may allow:

```
Developer
↓
main
```

This usually requires:

- excellent automated tests
- strong CI
- very small changes
- strong engineering discipline

Approach 2: Very short-lived branches

This is extremely common.

```
main
|
└─ feature/payment-validation
    ↓
    few commits
    ↓
    PR
    ↓
    merge quickly
```

The important principle is not:

No branches

It is:

No long-lived branches

4.4 What If a Feature Takes Three Weeks?

Do not necessarily keep a feature branch alive for three weeks.

Split the feature into small, safe increments.

Example: building a new checkout system.

Day 1

Add new CheckoutService

Merge it.

Day 2

```
Add pricing calculation
```

Merge it.

Day 3

```
Add new API endpoint
```

Merge it.

Day 4

```
Add new UI
```

Merge it.

The incomplete code can exist in `main` without being exposed to users by using **feature flags**.

4.5 Feature Flags

A feature flag decides whether functionality is active.

Simplified example:

```
if (newCheckoutEnabled) {  
    useNewCheckout();  
} else {  
    useOldCheckout();  
}
```

While development continues:

```
newCheckoutEnabled = false
```

Code can still be merged into `main`, but users do not see the unfinished feature.
Once everything is ready:

```
newCheckoutEnabled = true
```

This separates two concepts:

```
Code deployment
```

from:

```
Feature release
```

4.6 Deployment vs Release

Deployment

Code reaches the production infrastructure.

Release

Users actually receive access to the functionality.

A feature can therefore be:

```
Deployed = Yes  
Released = No
```

Feature flags make this possible.

4.7 Why Trunk-Based Development Works Well With CI

CI means **Continuous Integration**.

The important word is:

| Integration

Every merge can trigger:

```
Build
↓
Unit tests
↓
Integration tests
↓
Static analysis
↓
Security checks
```

This creates:

```
Trunk-Based Development
+
CI
↓
Frequent small validated integrations
```

Frequent integration generally reduces the amount of divergence that builds up between developers.

5. Gitflow vs GitHub Flow vs Trunk-Based Development

Area	Gitflow	GitHub Flow	Trunk-Based Development
Main production branch	<code>main</code>	<code>main</code>	trunk / <code>main</code>
<code>develop</code> branch	Yes	No	No
Feature branches	Yes	Yes	Optional / very short-lived

Area	Gitflow	GitHub Flow	Trunk-Based Development
Release branches	Yes	Usually no	Usually no
Hotfix branches	Yes	Usually normal fix branch	Usually small trunk fix
Branch lifetime	Can be long	Prefer short	Very short
Release style	Versioned / planned	Frequent deployment	Continuous integration
Complexity	High	Moderate	Process discipline is high
Best fit	Traditional release cycles	Web/SaaS teams	Strong CI/CD teams

Simple Mental Model

```

Gitflow
= structured release branches

GitHub Flow
= main + short-lived branches + PRs

Trunk-Based
= integrate into trunk very frequently

```

6. Pull Request Workflow

Pull Request is not a Git feature.

Git itself can merge branches without any Pull Request.

Example:

```

git switch main
git merge feature/payment
git push

```

Technically, that is enough for Git.

A Pull Request adds a collaboration and review layer around the merge.

It gives the team a place to:

- review code
- discuss changes
- run automated checks
- request changes
- approve the change
- enforce branch protection rules

6.1 Complete Pull Request Flow

Suppose you are building payment validation.

Start from the latest `main`:

```
git switch main
git pull origin main
```

Create a feature branch:

```
git switch -c feature/payment-validation
```

Make changes and commit:

```
git add .
git commit -m "Add payment validation"
```

Push:

```
git push -u origin feature/payment-validation
```

The remote repository now contains:

```
origin/main
origin/feature/payment-validation
```

Create a PR:

```
feature/payment-validation
    ↓
main
```

Then the professional workflow becomes:

```
Open PR
    ↓
Automated checks run
    ↓
Reviewer reads code
    ↓
Reviewer comments
    ↓
Developer fixes issues
    ↓
Push new commits
    ↓
PR updates automatically
    ↓
Review approved
    ↓
Merge
```

A Pull Request is therefore not just a request to combine branches. It is a **review and collaboration workflow around a proposed merge**.

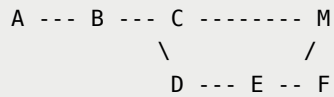
7. Ways to Merge a Pull Request

Three common strategies are:

1. Merge Commit
2. Squash and Merge
3. Rebase and Merge

7.1 Merge Commit

Git creates an additional merge commit.



Where:

M = merge commit

Advantages

- preserves original branch history
- clearly shows where the branch was merged
- keeps the branch topology visible

Disadvantage

- history can become noisy with many feature branches

7.2 Squash and Merge

Suppose a branch has:

```
D = start validation
E = fix typo
F = fix test
G = another fix
```

Instead of keeping every small commit in `main`, the platform combines them into a single commit.

Before:

```
D --- E --- F --- G
```

After squash:

```
S
```

Main becomes:

```
A --- B --- C --- S
```

Where:

```
S = complete payment validation
```

Benefit

The main branch gets a cleaner history with one meaningful commit for the complete feature.

This is very common in Pull Request workflows.

7.3 Rebase and Merge

The commits from the feature branch are replayed on top of the latest `main`.

Before:

```
A --- B --- C
          \
           D --- E
```

After:

```
A --- B --- C --- D' --- E'
```

There is no merge commit.

Result

- linear history
 - individual commits are preserved
 - commit IDs change because commits are recreated during rebase
-

7.4 When Should the Feature Branch Be Deleted?

Usually after the PR has been successfully merged.

```
feature/payment-validation
```

has completed its purpose.

Deleting finished branches helps keep the repository clean.

8. Code Review Etiquette

Code review is not only:

| A senior developer checking whether the code works.

A useful review may evaluate:

- correctness
- readability
- maintainability
- security
- performance
- testing
- architecture
- consistency

There is also a human side to review.

Review comments should be about the **code**, not the person who wrote it.

A good reviewer should explain **why** a change is being requested whenever possible.

8.1 Useful Review Comment Categories

Many teams informally use labels such as these.

Blocking

Something that should be fixed before merging.

Blocking:

This endpoint allows access without checking the user's role.

We should fix this before merge.

Suggestion

An improvement that is useful but may not block the PR.

Suggestion:

Could this validation be extracted into a helper method? It may make the service easier to read.

Question

The reviewer wants more context.

Question:

Why are we loading the user again here?
Could we reuse the existing entity?

Nit

A very small preference.

`nit` means **nitpick**.

```
Nit:
```

```
Maybe rename `p` to `payment`.
```

A nit normally should not block an otherwise good PR.

8.2 PR Size Matters

A huge PR such as:

```
92 files changed
+11,300
-6,700
```

is difficult to review properly.

A smaller PR such as:

```
4 files changed
+180
-40
```

is much easier to understand.

Professional Habit

| Small PRs are easier to review than giant PRs.

8.3 One PR Should Ideally Have One Purpose

Avoid mixing unrelated changes.

Bad

```
Payment feature
+
User refactoring
+
Logging changes
+
Database cleanup
+
CSS redesign
```

all inside one PR.

Better

```
PR 1
Add payment validation

PR 2
Refactor payment repository

PR 3
Add payment logging
```

Focused PRs are easier to:

- understand
- review
- test
- revert
- maintain

9. Monorepo vs Polyrepo

Branching strategy answers:

| How do we organize branches inside a repository?

Repository strategy answers:

| How many repositories should the system use?

Suppose an e-commerce platform contains:

- Frontend
- User Service
- Product Service
- Order Service
- Payment Service
- Notification Service

Two common approaches are:

Monorepo
Polyrepo

10. Polyrepo

Poly means **many**.

Different applications or services live in separate repositories.

Example:

```
github.com/company/frontend  
github.com/company/user-service  
github.com/company/order-service  
github.com/company/payment-service  
github.com/company/notification-service
```

Conceptually:

User Service
↓
Repository A

```
Payment Service
  ↓
Repository B

Order Service
  ↓
Repository C
```

10.1 Why Use Polyrepo?

A service may have its own team and need:

- its own CI/CD
- independent releases
- separate permissions
- separate issues
- independent repository history

For example:

```
payment-service → v4.2
order-service   → v8.1
```

Each service can evolve independently.

10.2 Benefits of Polyrepo

Clear Ownership

```
Payments Team
  ↓
payment-service repo
```

Independent Deployment

A change to payment can deploy only the payment service.

Smaller Repositories

Developers clone only the repository they need.

Easier Repository-Level Permissions

Access can be controlled separately for individual repositories.

10.3 Polyrepo Challenges

Cross-service changes require coordination.

Suppose an API field changes from:

```
amount
```

to:

```
amountInPaise
```

The change may affect:

- frontend repository
- payment-service repository
- order-service repository

Instead of one PR, the team may need:

```
PR in frontend  
PR in payment  
PR in order
```

These PRs must be coordinated.

10.4 Shared Code in Polyrepo

Suppose several Java services need:

- common logging
- common authentication utilities
- common DTOs

That shared code may need its own library or repository, for example:

```
company-common
```

This introduces versioning and dependency management between repositories.

11. Monorepo

Mono means **one**.

Multiple applications or services are stored inside one Git repository.

Example:

```
ecommerce-platform/  
|  
├─ frontend/  
├─ user-service/  
├─ order-service/  
├─ payment-service/  
├─ notification-service/  
└─ shared/
```

Everything lives inside:

```
ONE Git repository
```

11.1 Monorepo Does Not Mean Monolith

This distinction is important.

You can have:

```
20 microservices
```

inside:

```
1 Git repository
```

and still have a microservice architecture.

These are separate decisions:

```
Software architecture  
= how the system runs  
  
Repository strategy  
= how the source code is stored
```

Therefore both are valid:

```
Microservices + Monorepo
```

and:

```
Microservices + Polyrepo
```

Likewise, a monolithic application could also be stored using more than one repository if a team deliberately structures its code that way.

11.2 Why Monorepo Can Be Powerful

Suppose one change affects:

- Payment API
- Order Service

- Frontend

In polyrepo:

```
PR 1
PR 2
PR 3
```

In a monorepo:

```
ONE branch
  ↓
ONE PR
  ↓
frontend/
payment-service/
order-service/
```

All related changes can move together.

This enables an **atomic change**:

| Related source-code changes become part of the repository together.

11.3 Shared Code in a Monorepo

Shared code can live inside the same repository.

```
shared/
├─ auth/
├─ logging/
└─ utilities/
```

Multiple applications can use it.

11.4 Monorepo Challenges

Large monorepos require stronger tooling.

Suppose a repository contains:

```
200 applications
15 million lines of code
```

A small change should not require rebuilding everything.

Tooling needs to determine:

```
What changed?
  ↓
Which projects depend on it?
  ↓
What should be built and tested?
```

So a monorepo can simplify source-code coordination while making CI/CD and repository tooling more complex.

Other challenges can include:

- repository size
- clone performance
- large dependency graphs
- build orchestration
- permission boundaries
- scaling developer tooling

12. Monorepo vs Polyrepo

Area	Monorepo	Polyrepo
Number of Git repositories	One	Many
Cross-project changes	Easier to make atomically	Require coordination
Shared code	Easier to keep together	Usually separate package/repo

Area	Monorepo	Polyrepo
CI/CD tooling	Can become complex at scale	More isolated
Ownership	Shared repository boundaries	Clear repo-level ownership
Permissions	Harder to isolate at repo level	Easier per repository
Clone size	Can become very large	Usually smaller
Independent releases	Possible, but tooling is needed	Natural
Best fit	Closely related projects / strong tooling	Independent teams/services

13. Signed Commits

A normal Git commit may show:

```
Author: Aditya Tandon <aditya@example.com>
```

That does **not** automatically prove that the real Aditya created the commit.

Git allows anyone to configure author metadata:

```
git config user.name "Aditya Tandon"
git config user.email "aditya@example.com"
```

Someone could also write:

```
git config user.name "Linus Torvalds"
```

The author name and email stored in a normal commit are metadata.

They are not strong cryptographic identity verification.

13.1 Why Commit Signing Exists

Signed commits use public-key cryptography.

A developer controls:

Private Key
Public Key

The private key remains secret.

The public key can be shared.

Conceptually:

Commit data
+
Private key
↓
Digital signature

Someone can then use the corresponding public key to verify that signature.

Signing helps prove that the commit was signed using a particular private key.

13.2 Does Signing Encrypt the Commit?

No.

Signing does **not** hide the source code.

It helps provide:

- authenticity
- integrity

The commit contents remain readable.

13.3 Common Commit-Signing Methods

Git can work with signing mechanisms such as:

- GPG
- SSH
- S/MIME

Conceptually, a signed commit can be created with:

```
git commit -S -m "Add payment validation"
```

- `-S` tells Git to sign the commit.

The exact configuration depends on the signing mechanism being used.

13.4 Where Signed Commits Matter

Signed commits are especially useful for:

- open-source maintainers
 - security-sensitive repositories
 - enterprise compliance
 - software supply-chain security
 - release verification
 - protected repositories and branches
-

14. Final Mental Model

Branching Strategy

How should developers create, use and merge branches?

Gitflow

```
main  
develop  
feature/*  
release/*  
hotfix/*
```

Best suited to structured release cycles.

GitHub Flow

```
main
↓
short-lived branch
↓
PR
↓
review + CI
↓
merge
```

Simple and effective for frequently deployed applications.

Trunk-Based Development

```
small change
↓
integrate quickly
↓
small change
↓
integrate quickly
```

The goal is to minimize long-lived divergence.

Pull Requests

```
Proposed change
↓
Automated checks
↓
Code review
↓
Discussion / fixes
↓
Approval
↓
Merge
```

A PR is a collaboration workflow provided by hosting platforms, not a core Git feature.

Repository Strategy

Monorepo
= many projects in one repository

Polyrepo
= projects/services split across repositories

Repository structure and software architecture are separate decisions.

Signed Commits

Normal author metadata
≠
cryptographic proof

Signed commits add cryptographic verification to the commit workflow.

Quick Revision Cheat Sheet

Concept	Remember
Branching strategy	Rules for creating, merging and managing branches
Gitflow	<code>main</code> + <code>develop</code> + feature/release/hotfix branches
GitHub Flow	<code>main</code> + short-lived branches + PRs
Trunk-Based	Integrate very frequently; avoid long-lived branches
Feature flag	Merge code without exposing unfinished functionality
Deployment	Code reaches production
Release	Users receive the feature
Pull Request	Review/collaboration layer around a proposed merge

Concept	Remember
Merge commit	Preserves branch topology
Squash merge	Combines feature commits into one
Rebase merge	Replays commits for linear history
Small PR	Easier to review and reason about
Monorepo	Many projects in one Git repository
Polyrepo	Projects/services split across repositories
Signed commit	Cryptographically verifies a signature associated with a commit